

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Notas de aula: ED para conjuntos disjuntos \(Union-find\).](#)

Uma ED para conjuntos disjuntos mantém uma coleção $\{S_1, S_2, \dots, S_k\}$ de conjuntos disjuntos, e suporta as operações:

MAKE-SET(x):
cria novo conjunto $\{x\}$.

UNION(x,y):
une os conjuntos (disjuntos) que contém x e y , digamos S_x e S_y , em um novo conjunto $S_x \cup S_y$. Os conjuntos S_x e S_y são descartados.

FIND-SET(x):
retorna ponteiro para elemento representante do (único) conjunto que contém x .

Exemplo de aplicação: encontrar componentes conexos em grafo não direcionado.

```
COMPONENTES(Adj,n):
  para cada vértice v, faça MAKE-SET(v)  O(V) chamadas MAKE-SET
  para cada aresta (u,v),                O(E)
    se FIND-SET(u) != FIND-SET(v)        O(E) chamadas FIND-SET
      UNION(u,v)                        O(V) chamadas UNION

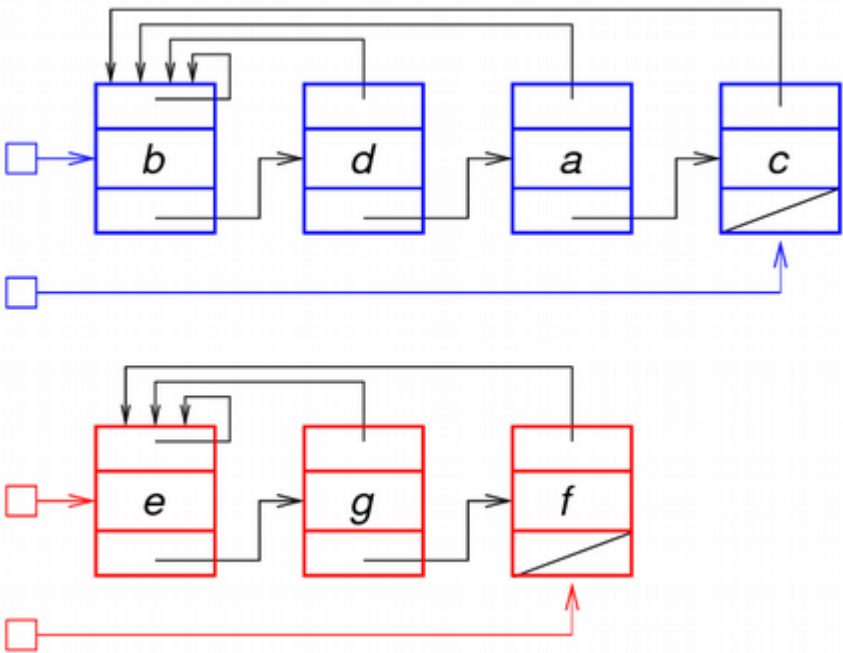
MESMO-COMPONENTE(u,v):
  retorne (FIND-SET(u) == FIND-SET(v))  O(1) chamadas FIND-SET
```

Vamos medir a complexidade de uma sequência de operações MAKE-SET, UNION e FIND-SET.

$$\underbrace{M \ M \ M}_n \underbrace{U \ F \ U \ U \ F \ U \ F \ F \ U \ F}_m$$

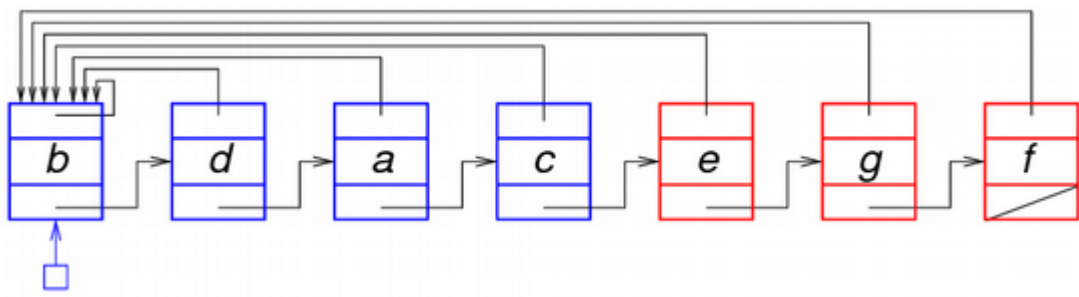
Uma operação isoladamente pode ter complexidade alta, mas queremos que a complexidade da sequência seja baixa (análise amortizada).

Implementação com lista encadeada



O primeiro nó é o representante. Existe ponteiro para o último.
Cada elemento aponta para o representante.

MAKE-SET(x): $O(1)$. FIND-SET(x): $O(1)$.
UNION(x,y) concatena duas listas e gasta $O(n)$, pois é necessário atualizar ponteiros para o representante.



Operação	Número de atualizações
MAKE-SET(x_1)	1
MAKE-SET(x_2)	1
$\vdots$	$\vdots$
MAKE-SET(x_n)	1
UNION(x_1, x_2)	1
UNION(x_2, x_3)	2
UNION(x_3, x_4)	3
$\vdots$	$\vdots$
UNION(x_{n-1}, x_n)	$n-1$

Quantidade de operações: $2n - 1 \in \Theta(n)$.

Custo total: $O(n^2)$.

Custo amortizado por operação: $O(n)$.

Podemos melhorar colocando a menor lista no final da maior.

Se atualizamos o representante de x , então x está na menor lista, e portanto a lista que contém x pelo menos dobrar tamanho em cada atualização. Portanto, o número de atualização de representante de um elemento é $O(\lg n)$.

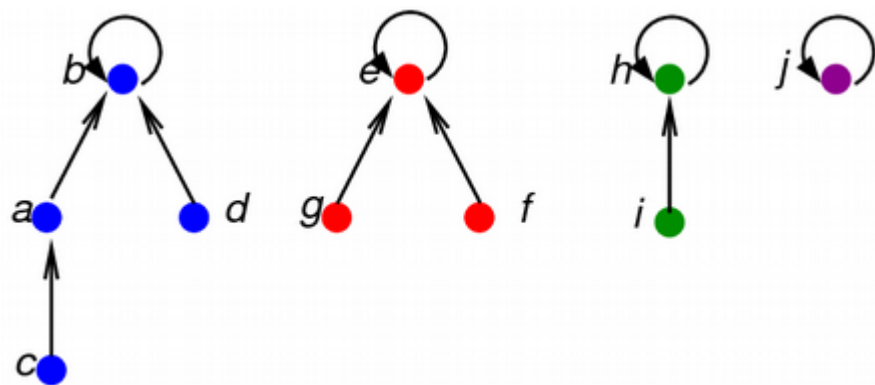
Assim, o custo para realizar todas as atualizações de representantes é $O(n \lg n)$. Retirando o custo de atualizar os representante, o que resta em cada uma das m operações pode ser feito em $O(1)$. Portanto, o custo total será $O(m + n \lg n)$.

Se $m \gg n$, então o custo amortizado por operação será $O(1)$.

Se $m \in O(n)$, então cada operação terá custo amortizado $O(\lg n)$.

Obs.: como o máximo de operações UNION é $n - 1$, para ter $m \gg n$ teríamos que fazer muitas operações FIND-SET. comum é ter $m \in O(n)$.

Implementação com floresta de conjuntos disjuntos



Cada conjunto é uma árvore enraizada.

Cada elemento aponta para o pai.

A raiz é o representante, e aponta para si mesma.

MAKE-SET(x):

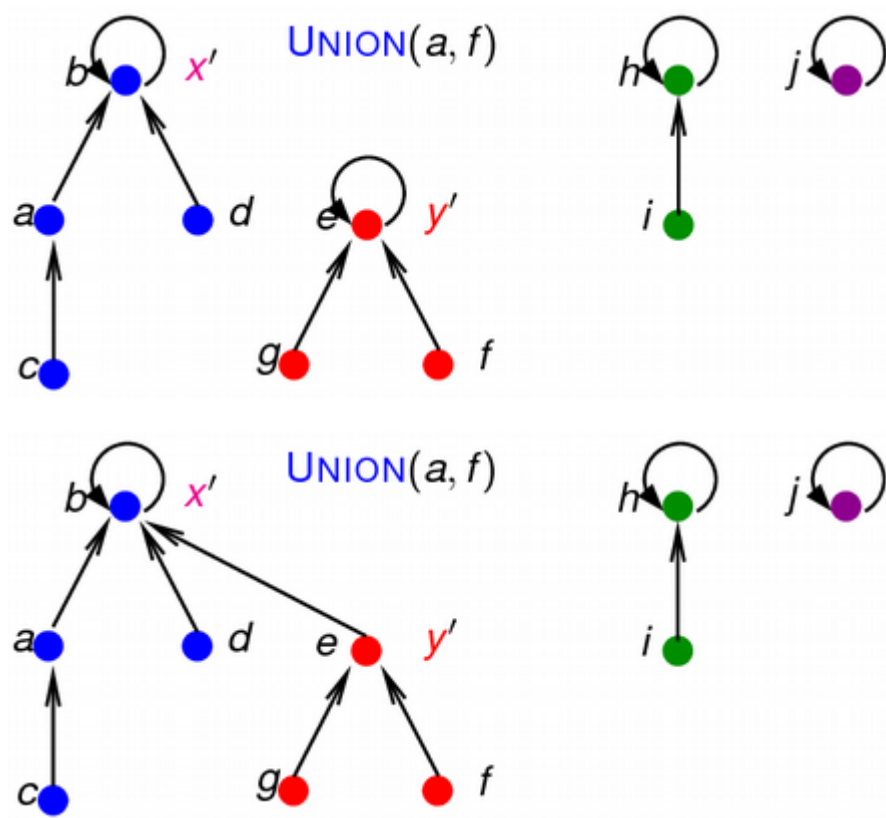
```
pai[x] <- x
```

FIND-SET(x):

```
se pai[x] = x, retorne x
retorne FIND-SET(pai[x])
```

UNION(x, y):

```
x' <- FIND-SET(x)
y' <- FIND-SET(y)
pai[y'] <- x'
```



A complexidade não é melhor, pois no pior caso cada árvore é uma lista encadeada, e localizar o representante ser

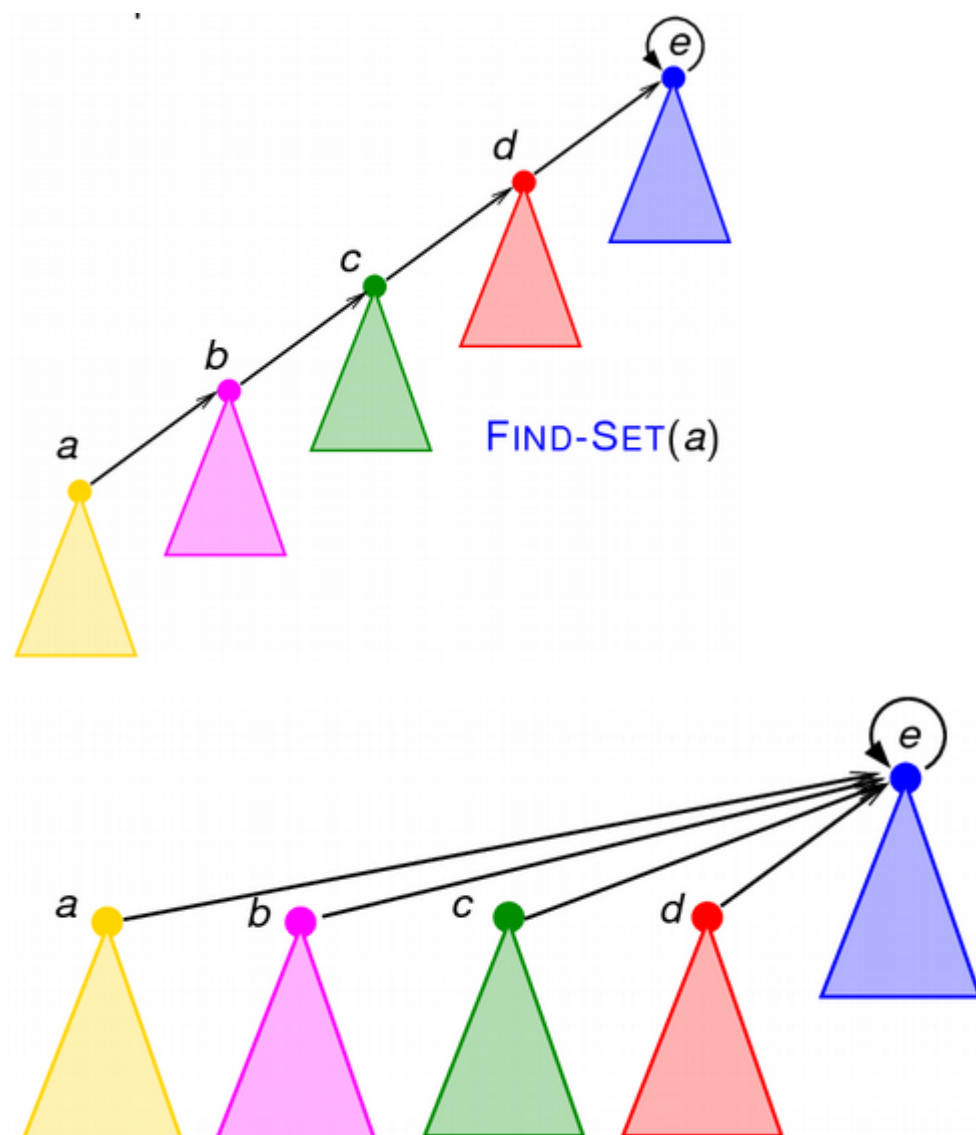
As heurísticas *union by rank* e *path compression* melhoram a complexidade, embora a análise não seja simples.

Union by rank: na união colocamos a menor árvore como filha da maior árvore. `rank[]` fornece estimativa da altura da árvore.

```
MAKE-SET(x):
    pai[x] <- x
    rank[x] <- 0

UNION(x,y):
    x' <- FIND-SET(x)
    y' <- FIND-SET(y)
    se rank[x'] > rank[y'], então pai[y'] <- x'
    senão, pai[x'] <- y'
        se rank[x'] = rank[y']
            rank[y'] <- rank[y'] + 1
```

Path compression: quando o representante passar por um nó nas chamadas recursivas de FIND-SET, aproveite e torne-o filho do representante.



```
FIND-SET(x):
    se pai[x] != x
        pai[x] <- FIND-SET(pai[x])
    retorne pai[x]
```

Análise de complexidade

Vou mostrar um esboço da prova de que uma sequência de m operações MAKE-SET, UNION e FIND-SET pode ser executada em tempo $O(m \lg^*(n))$ quando usamos floresta de conjuntos disjuntos com *union by rank* e *path compression*. Uma demonstração mais detalhada pode ser encontrada no Cormen.

A função $\lg^*(n)$ é o número de vezes que aplicamos a função $\lg$ para tornar o valor n menor ou igual a 1. Por exemplo, $\lg^*(1000) = 4$ pois $\lg(\lg(\lg(1000))) > 1$ e $\lg(\lg(\lg(\lg(1000)))) \leq 1$. Esta função cresce muito lentamente: para valores do que 5, n teria que ser maior que $2^{65536} = (2^4)^{16384} > 10^{16384}$. Note que o número de partículas no universo visível é de 10^{80} . Portanto, em termos práticos podemos tratar $\lg^*(n)$ como uma constante. Assim, o custo amortizado por operação é $O(1)$.

Para simplificar a análise vamos dividir cada operação UNION em duas operações FIND-SET e uma operação LINK, como descrito abaixo. Esta divisão triplica cada operação UNION, mas isso não afeta a complexidade: continua sendo $O(m \lg^*(n))$. Cada operação MAKE-SET e LINK executa em $O(1)$, restando analisar a operação FIND-SET.

```
UNION(x,y):
    x' <- FIND-SET(x)
    y' <- FIND-SET(y)
    LINK(x',y')

LINK(x',y'):
    se rank[x'] > rank[y'], então pai[y'] <- x'
    senão, pai[x'] <- y'
        se rank[x'] = rank[y']
            rank[y'] <- rank[y'] + 1
```

As propriedades abaixo referem-se aos ranks dos nós. Observe que o rank é atualizado apenas nas raízes e na operação LINK, e que um nó que deixa de ser raiz não volta a ser raiz.

Propriedade 1: Para qualquer nó x não raiz, $\text{rank}[x] < \text{rank}[\text{pai}[x]]$.

Vamos analisar as duas situações em que ocorre atualização do pai de um nó: na operação LINK e no path compression.

Na operação LINK, quando a raiz com menor rank aponta para a raiz de maior rank, o novo pai terá rank maior. Isto ocorre quando as duas raízes possuem o mesmo rank, pois o novo pai sofrerá incremento no rank.

Se a propriedade vale antes do path compression então valerá depois do path compression, pois o pai é trocado pelo ancestral mais próximo da raiz, que por transitividade também tem rank maior.

Propriedade 2: No instante que o rank de um nó é incrementado de $k - 1$ para k , sua árvore tem pelo menos 2^k nós.

Por indução em k .

Base: se $k = 1$ então temos a união de duas folhas, e a árvore possui $2^1 = 2$ nós.

Passo indutivo:

Enquanto um nó é raiz ele não perde descendentes devido ao path compression. Quando o rank aumentou de $k - 1$ houve a união de duas árvores com rank $k - 1$ na raiz. Portanto, por hipótese de indução cada uma destas árvores tem pelo menos 2^{k-1} nós. Assim, o total de nós vale pelo menos $2 \cdot 2^{k-1} = 2^k$.

Propriedade 3: Ao final do algoritmo, existe no máximo $n/2^k$ nós com rank k .

Pela Propriedade 2, quando um nó teve seu rank incrementado para k sua árvore ficou com pelo menos 2^k nós. Observe que estamos considerando o rank final de cada nó.

Vamos considerar como descendentes de um nó de rank k aqueles nós que eram seus descendentes no instante que o rank foi incrementado para k (pois o path compression pode remover descendentes de nós não raízes).

Pela Propriedade 1 cada nó tem no máximo um ancestral com rank k . Então não existe compartilhamento de descendentes entre nós distintos de mesmo rank.

Como temos pelo menos 2^k nós em cada conjunto formado por um nó de rank k e seus descendentes, e estes conjuntos são disjuntos, concluímos que existe no máximo $n/2^k$ nós com rank k .

Vamos fazer a seguir uma *análise amortizada com função potencial*. Cada nó pode receber algumas moedas ao longo do algoritmo. Ao executar um FIND-SET, algumas operações serão contabilizadas e outras serão pagas com moedas. Precisamos mostrar que um nó sempre tem moedas disponíveis para pagar as operações, e que o total de operações somado com o total de moedas recebidas é $O(m \lg^* n)$.

Pela Propriedade 3 temos que os ranks vão de 0 até $\lg n$. Vamos particionar os ranks positivos em intervalos do tipo $\{k + 1, k + 2, \dots, 2^k\}$:

$$\{1\}, \{2\}, \{3, 4\}, \{5, 6, \dots, 16\}, \{17, 18, \dots, 2^{16} = 65536\}, \{65537, \dots, 2^{65536}\}, \dots$$

O 1o intervalo contém os elementos i com $\lg^*(i) = 0$, o 2o intervalo contém os elementos i com $\lg^*(i) = 1$, assim sucessivamente. Portanto, o número de intervalos vale $\lg^*(n) + 1$.

Quando um nó deixa de ser raiz, se seu rank pertencer ao intervalo $\{k + 1, k + 2, \dots, 2^k\}$ receberá 2^k moedas. Pela Propriedade 3, o total de nós com rank neste intervalo é no máximo

$$\begin{aligned} \frac{n}{2^{k+1}} + \frac{n}{2^{k+2}} + \dots + \frac{n}{2^{2^k}} &\leq \frac{n}{2^{k+1}} + \frac{n}{2^{k+2}} + \dots \\ &= \frac{n}{2^{k+1}} \sum_{i=0}^{\infty} \frac{1}{2^i} \\ &= \frac{n}{2^{k+1}} \cdot 2 = \frac{n}{2^k}. \end{aligned}$$

Portanto, o total de moedas distribuídas em cada intervalo vale $(n/2^k) \cdot 2^k = n$. Temos então que o total de moedas distribuídas vale $n(\lg^* n + 1) \in O(n \lg^* n)$.

Sempre que o FIND-SET passar por um nó i consideramos dois casos: (1) se o rank do pai de i estiver em um intervalo diferente do rank de i computamos o gasto de uma operação; (2) caso contrário, gastamos uma das moedas de i .

Como as moedas são fornecidas depois do nó deixar de ser raiz, ele não muda mais de rank e nem de intervalo. Como o FIND-SET realiza em cada nó i não raiz uma troca de pai por outro com rank maior, o máximo de moedas gastadas nele é 2^k , pois quando o rank do pai sair do intervalo o nó i permanecerá no caso (1). Concluímos que o total de moedas distribuídas ($O(n \lg^* n)$ moedas) é suficiente para realizar todos os pagamentos.

Em uma chamada FIND-SET o número de nós no caso (1) é $O(\lg^* n)$, pois o rank é crescente e o número de pares de intervalos consecutivos é da ordem do número de intervalos. Portanto, o total de operações contabilizadas no caso (1) é $O(m \lg^* n)$.

Concluímos que o custo total das chamadas FIND-SET é $O(m \lg^* n) + O(n \lg^* n) \in O(m \lg^* n)$.

◀ [Vídeo: tabelas hash \(12min\)](#)

Seguir para...

[Grafos: conceitos e representação \(C++\)](#) ▶

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

📱 [Baixar o aplicativo móvel.](#)